

kantorku

Panduan Pemasangan dan Konfigurasi Lengkap

Kantor digital yang sesungguhnya

AI Worker Orchestration Framework v0.1.0

Manager terima brief dari client, diskusi dengan tim,
tim riset sudah mulai kerja sebelum disuruh,
dan para coder tinggal verifikasi + edit konteks yang sudah disiapkan.

1. Persyaratan Sistem

Sebelum memasang kantorku, pastikan sistem Anda memenuhi persyaratan minimum berikut. Kantorku membutuhkan Python 3.11 atau yang lebih baru karena menggunakan fitur-fitur bahasa modern seperti type hints, union types, dan async generators yang tidak tersedia di versi Python sebelumnya. Selain itu, kantorku bergantung pada beberapa library inti yang akan diinstal secara otomatis melalui pip.

1.1 Prasyarat Perangkat Lunak

Komponen	Versi Minimum	Catatan
Python	3.11+	Wajib. Gunakan python3 --version untuk cek
pip	24.0+	Manajer paket Python
Git	2.30+	Opsional, untuk clone repository
Ollama	Terbaru	Opsional, untuk LLM lokal (gratis)

1.2 API Keys (Opsional)

Kantorku mendukung 5 provider LLM. Anda tidak perlu semua API key sekaligus — cukup provider yang ingin Anda gunakan. Untuk memulai tanpa biaya, gunakan Ollama (lokal, gratis). Setiap provider dipasang sebagai dependency opsional, sehingga Anda hanya menginstal yang benar-benar dibutuhkan tanpa membebani lingkungan Python Anda.

Provider	Env Variable	Install Command	Harga
Anthropic	ANTHROPIC_API_KEY	pip install kantorku[anthropic]	\$3-15/1M token
Google	GOOGLE_API_KEY	pip install kantorku[google]	\$1.25-10/1M token
MiniMax	MINIMAX_API_KEY	pip install kantorku[minimax]	\$0.12-0.30/1M token
DeepSeek	DEEPSEEK_API_KEY	pip install kantorku[deepseek]	\$0.27-0.28/1M token
Ollama	-	pip install kantorku[ollama]	Gratis (lokal)

2. Pemasangan

Ada tiga cara memasang kantorku: dari PyPI (cara paling mudah), dari source (untuk development), dan menggunakan virtual environment (direkomendasikan untuk isolasi). Setiap metode menghasilkan instalasi yang identik secara fungsional — pilih yang paling sesuai dengan workflow Anda.

2.1 Metode A: Instal dari PyPI (Direkomendasikan)

Cara paling mudah dan cepat. Kantorku dipublikasikan di PyPI sehingga bisa diinstal langsung dengan pip. Gunakan opsi [all] untuk memasang semua provider LLM sekaligus, atau pilih hanya provider yang Anda butuhkan untuk menghemat ruang disk dan menghindari dependency yang tidak perlu.

Instal dengan semua provider:

```
pip install kantorku[all]
```

Instal hanya provider tertentu:

```
pip install kantorku[anthropic,deepseek]
```

Instal minimal (hanya core + Ollama):

```
pip install kantorku
```

2.2 Metode B: Instal dari Source

Jika Anda ingin berkontribusi atau mengubah kode kantorku, clone repository dan instal dalam mode editable. Mode editable memungkinkan perubahan kode langsung berlaku tanpa perlu reinstallasi. Pastikan hatchling build system terpasang karena kantorku menggunakannya sebagai build backend.

```
git clone https://github.com/your-org/kantorku.git
cd kantorku
pip install -e ".[dev]"
python tests/test_office.py # verifikasi
```

2.3 Metode C: Virtual Environment (Best Practice)

Sangat disarankan menggunakan virtual environment untuk mengisolasi dependency kantorku dari proyek Python lainnya. Ini mencegah konflik versi dan memastikan lingkungan yang bersih. Python 3.11+ sudah menyertakan venv sebagai modul bawaan, sehingga Anda tidak perlu menginstal alat tambahan.

```
python3 -m venv kantorku-env
source kantorku-env/bin/activate    # Linux/macOS
# atau:
kantorku-env\Scripts\activate      # Windows

pip install kantorku[all]
```

2.4 Verifikasi Instalasi

Setelah instalasi, verifikasi bahwa kantorku terpasang dengan benar dengan mengimpor modul utama dan menjalankan test suite bawaan. Jika semua impor berhasil tanpa error dan test suite menampilkan "ALL TESTS PASSED", maka kantorku siap digunakan.

```
python3 -c "from kantorku import Office; print('kantorku OK!')"
# Output: kantorku OK!

python3 -m pytest tests/ -v # jalankan semua test
```

3. Konfigurasi

Kantorku dikonfigurasi melalui file `kantorku.toml` yang mendefinisikan model LLM untuk setiap worker, pengaturan context pool, kredensial provider, dan path penyimpanan memory. File ini adalah pusat kendali seluruh operasi kantorku — setiap worker, pool instance, dan memory tier dapat dikustomisasi dari satu file tunggal ini.

3.1 Membuat File Konfigurasi

Buat file `kantorku.toml` di root directory proyek Anda. Anda bisa menyalin template dari repository kantorku atau membuat dari nol. File ini menggunakan format TOML yang mudah dibaca manusia dan didukung luas oleh editor kode modern.

```
cp kantorku.toml.example kantorku.toml
# atau buat manual:
```

3.2 Struktur Konfigurasi Lengkap

Berikut adalah struktur lengkap file `kantorku.toml` dengan penjelasan setiap section. Setiap section mengontrol aspek berbeda dari operasi kantorku, mulai dari model Conductor hingga pengaturan server.

Section [office] — Conductor Model

```
[office]
conductor_model = "anthropic/claude-opus-4-6"
# Model untuk Conductor (CEO).
# Hanya untuk decision-making + orchestration.
# Mahal ($5/M) tapi frekuensi pakai rendah.
```

Section [workers.*] — Worker Definitions

```
[workers.coder_frontend]
model = "anthropic/claude-sonnet-4-6"
squad = "coding"
role = "React/CSS/UI/Visual specialist"

[workers.coder_backend]
model = "minimax/minimax-m2-7"
squad = "coding"
role = "Python/Rust/Systems specialist"

[workers.coder_wiring]
model = "google/gemini-3-1-pro"
squad = "coding"
role = "API/WS/MCP/Glue specialist"
```

Section [pool] — DeepSeek Context Pool

```
[pool]
model = "deepseek/deepseek-v3-2"
instances = 3
queue_type = "fifo"
# 3 instance standby, FIFO queue.
# Proaktif saat briefing, reaktif saat diminta.
```

Section [providers.*] — API Credentials

```
[providers.anthropic]
api_key = "${ANTHROPIC_API_KEY}"

[providers.google]
api_key = "${GOOGLE_API_KEY}"

[providers.minimax]
api_key = "${MINIMAX_API_KEY}"

[providers.deepseek]
api_key = "${DEEPSEEK_API_KEY}"

[providers.ollama]
base_url = "http://localhost:11434/v1"
```

Tip: Gunakan format `${ENV_VAR}` untuk merujuk environment variable. Kantorku akan otomatis meresolve saat runtime, sehingga API key tidak perlu ditulis langsung di file konfigurasi.

Section [memory] — Three-Ring Memory

```
[memory]
ring1_path = "data/ring1.duckdb" # Hot: DuckDB
ring2_path = "data/ring2.db"     # Warm: SQLite
ring3_enabled = false           # Cold: Cognee (Fase 3)
ring3_path = "data/ring3"
```

4. Setting API Keys

API keys adalah kredensial yang memungkinkan kantorku berkomunikasi dengan berbagai provider LLM. Keamanan API key sangat penting — jangan pernah menyimpannya langsung di file konfigurasi atau source code. Kantorku mendukung referensi environment variable dengan sintaks `${VAR_NAME}` di `kantorku.toml`, yang secara otomatis diresolve saat runtime.

4.1 Metode Environment Variable

```
export ANTHROPIC_API_KEY="sk-ant-..."
export GOOGLE_API_KEY="AIza..."
export MINIMAX_API_KEY="mmx-..."
export DEEPSEEK_API_KEY="sk-..."
```

4.2 Menggunakan .env File

Untuk kemudahan development, Anda bisa menggunakan file `.env` yang dimuat secara otomatis. File ini tidak boleh di-commit ke Git — tambahkan ke `.gitignore` untuk mencegah kebocoran kredensial. Gunakan `python-dotenv` atau shell sourcing untuk memuat variabel dari file ini sebelum menjalankan kantorku.

```
# .env (JANGAN commit ke Git!)
ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_API_KEY=AIza...
MINIMAX_API_KEY=mmx-...
DEEPSEEK_API_KEY=sk-...

# Muat sebelum menjalankan:
set -a && source .env && set +a
python -m kantorku.server
```

4.3 Setup Ollama (Gratis, Lokal)

Ollama adalah cara paling mudah untuk menjalankan LLM secara lokal tanpa API key. Ini sangat cocok untuk development dan testing karena sepenuhnya gratis dan data tidak pernah meninggalkan mesin Anda. Setelah Ollama terinstal, Anda tinggal pull model yang diinginkan dan menjalankan server.

```
# Instal Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Pull model
ollama pull llama3
ollama pull deepseek-coder-v2

# Verifikasi
ollama list
```

Ollama berjalan di `http://localhost:11434` secara default. Kantorku sudah dikonfigurasi untuk menggunakan endpoint ini secara otomatis melalui section `[providers.ollama]` di `kantorku.toml`.

5. Quick Start

Setelah konfigurasi selesai, Anda bisa mulai menggunakan kantorku dalam hitungan menit. Ada dua cara utama: programmatic API (untuk integrasi ke aplikasi Python Anda) dan server mode (untuk UI/frontend via WebSocket).

5.1 Programmatic API

Cara paling langsung — import Office dan mulai bekerja. API ini mirip dengan cara Anda menggunakan CrewAI atau LangGraph, tetapi dengan fitur-fitur unik kantorku seperti Contract flow, BriefingRoom, dan Context Pool.

```
import asyncio
from kantorku import Office

async def main():
    # 1. Buat office dari config
    office = Office.from_config("kantorku.toml")
    await office.initialize()

    # 2. One-shot run (paling simpel)
    result = await office.run(
        "Buat rate limiter di Rust"
    )
    print(result)

    # 3. Shutdown
    await office.shutdown()

asyncio.run(main())
```

5.2 Step-by-Step (Panel 1 + Panel 2)

Untuk kontrol lebih granular, gunakan alur step-by-step yang memisahkan fase negosiasi kontrak (Panel 1) dan eksekusi pekerjaan (Panel 2). Ini memungkinkan client memberikan feedback sebelum pekerjaan dimulai, dan mengamati progress secara real-time.

```
office = Office.from_config("kantorku.toml")
await office.initialize()

# Panel 1: Chat dengan Manager
async for event in office.chat(
    "session-1",
    "Buat rate limiter di Rust"
):
    if event["type"] == "manager_message":
        print(f"Manager: {event['content']}")
    elif event["type"] == "contract_ready":
```

```

        print(f"Kontrak: {event['contract']}")

# Client setuju → mulai kerja
result = await office.accept_and_run("session-1")
print(result)

```

5.3 Server Mode (WebSocket)

Jalankan kantorku sebagai server untuk terhubung dengan frontend (Tauri, React, dll). Server menyediakan dua channel WebSocket independen: `/ws/client` untuk Panel 1 dan `/ws/office` untuk Panel 2. Ini memungkinkan arsitektur dual-panel seperti yang dijelaskan dalam arsitektur kantorku.

```

# Jalankan server
kantorku --config kantorku.toml --port 8000

# atau dengan uvicorn langsung:
uvicorn kantorku.server:app --host 0.0.0.0 --port 8000

```

WebSocket Channel 1: `/ws/client`

Channel untuk interaksi user dengan Manager (Panel 1). Client mengirim pesan dan menerima respons dari Conductor, termasuk `contract_ready` event ketika kontrak siap.

```

# Client kirim:
{"type": "user_message", "content": "Buat rate limiter"}

# Server respons:
{"type": "manager_message", "content": "Production atau internal?"}
{"type": "contract_ready", "contract": {...}}

# Client setuju:
{"type": "contract_accepted", "contract": {...}}

```

WebSocket Channel 2: `/ws/office`

Channel untuk live event stream dari office (Panel 2). Semua aktivitas worker, context fetch, dan verification dikirim secara real-time ke channel ini, memungkinkan monitoring menyeluruh.

```

# Connect:
ws://localhost:8000/ws/office?session_id=session-1

# Events yang diterima:
{"type": "briefing_opened", "from": "conductor", ...}
{"type": "context_fetch_start", "instance": 0, ...}
{"type": "task_assigned", "to": "coder_backend", ...}
{"type": "task_done", "from": "coder_backend", ...}

```

6. Membuat Custom Worker

Kantorku dirancang untuk extensible — Anda bisa membuat worker kustom yang mewarisi BaseWorker dan mengimplementasikan method handle(). Worker kustom bisa menggunakan LLM manapun melalui self.llm_call(), mengakses konteks yang sudah di-prefetch melalui self.get_context(), dan berkomunikasi dengan worker lain melalui WorkerHub. Ini membuat kantorku sangat fleksibel untuk berbagai use case.

```
from kantorku import BaseWorker, Office
from kantorku.worker.base import Task, TaskResult

class SecurityAuditor(BaseWorker):
    """Custom worker untuk security audit."""

    async def handle(self, task: Task) -> TaskResult:
        # Ambil konteks yang sudah di-prefetch
        context = await self.get_context(task.id)

        prompt = f"""
        Audit keamanan untuk: {task.instruction}

        Konteks: {context or 'Tidak ada'}

        Periksa:
        1. SQL injection, XSS, CSRF
        2. Authentication/Authorization
        3. Input validation
        4. Sensitive data exposure
        """

        response = await self.llm_call(prompt)
        return TaskResult(
            task_id=task.id,
            status="done",
            output=response,
        )

# Daftarkan ke Office
office = Office()
office.hire_worker(
    "security_auditor",
    model="anthropic/claude-sonnet-4-6",
    squad="verification",
    role="Security audit specialist",
    worker_class=SecurityAuditor,
)
```

Worker kustom juga bisa meng-override method speak_up() untuk berpartisipasi dalam BriefingRoom, dan receive_dm() untuk menerima pesan langsung dari worker lain. Ini memungkinkan pola komunikasi yang kaya antar worker tanpa perlu mengubah kode inti

kantorku.

7. Tabel Worker dan Model

Berikut adalah tabel lengkap seluruh worker kantorku dengan model yang direkomendasikan berdasarkan benchmark terbaru April 2026. Setiap worker dipilih berdasarkan keunggulannya di benchmark spesifik — bukan berdasarkan popularitas atau harga semata. Strategi ini memastikan setiap tugas dikerjakan oleh model yang paling kompeten di domain tersebut.

Worker	Model	Squad	Harga/1M
conductor	Claude Opus 4.6	orchestration	\$5.00
intake	Llama 3 (Ollama)	translation	gratis
narrator	Llama 3 (Ollama)	translation	gratis
coder_frontend	Claude Sonnet 4.6	coding	\$1.50
coder_backend	MiniMax M2.7	coding	\$0.30
coder_wiring	Gemini 3.1 Pro	coding	\$2.00
verifier_designer	Gemini 3.1 Pro	verification	\$2.00
verifier_engineer	MiniMax M2.5	verification	\$0.12
pool (x3)	DeepSeek V3.2	context	\$0.28
debugger	DeepSeek V3.2	support	\$0.28
scout	Gemini 2.5 Pro	support	\$1.25
auditor	Claude Sonnet 4.6	support	\$1.50
scribe	DeepSeek V4 Flash	support	\$0.27
summarizer	DeepSeek V4 Flash	support	\$0.27
sentinel	Llama 3 (Ollama)	support	gratis

Rata-rata biaya per task berkisar \$0.50-\$2.00 tergantung kompleksitas, karena sebagian besar pekerjaan dilakukan oleh model murah (MiniMax, DeepSeek) sementara model mahal (Claude Opus) hanya digunakan untuk decision-making yang frekuensinya rendah.

8. Three-Ring Memory

Kantorku menggunakan sistem memory tiga lapis yang terinspirasi dari arsitektur CPU cache. Setiap ring memiliki karakteristik yang berbeda dalam hal kecepatan akses, kapasitas, dan persistensi, memungkinkan kantorku menyimpan data pada tier yang paling sesuai dengan pola aksesnya.

8.1 Ring 1 — DuckDB Hot Memory

Ring 1 menyimpan data yang paling sering diakses dan membutuhkan latensi terendah: konteks yang sudah di-prefetch oleh Context Pool, state session aktif, riwayat percakapan, dan hasil task terbaru. DuckDB berjalan in-process (tanpa network overhead) dengan latensi mikrodetik, menjadikannya ideal untuk data yang perlu diakses berkali-kali selama satu sesi kerja.

- Latensi: mikrodetik (in-process)
- Konten: prefetched context, session state, task results, history
- Kapasitas: sesuai RAM tersedia
- Persistensi: file .duckdb di disk

8.2 Ring 2 — SQLite Warm Memory

Ring 2 menyimpan data yang tidak perlu diakses setiap saat tetapi penting untuk analisis jangka panjang: episode log (apa yang terjadi, apa yang berhasil, apa yang gagal), pelajaran yang dipetik oleh Sentinel, dan audit trail untuk setiap aksi worker. Data ini berguna untuk fine-tuning di masa depan dan untuk memahami pola kerja tim dari waktu ke waktu.

- Latensi: milidetik
- Konten: episode logs, lessons learned, audit trail
- Format: SQLite + Parquet (untuk bulk analytics)
- Use case: training data, post-mortem analysis

8.3 Ring 3 — Cognee GraphRAG Cold Memory

Ring 3 adalah tier penyimpanan jangka panjang yang menggunakan Cognee untuk GraphRAG (Graph-based Retrieval Augmented Generation). Ini memungkinkan pencarian semantik lintas sesi dan pembentukan knowledge graph dari semua histori pekerjaan. Ring 3 masih dalam tahap stub dan akan diimplementasikan di Fase 3 development kantorku.

- Latensi: ratusan milidetik
- Konten: knowledge graph, semantic search index

- Use case: cross-session learning, pattern discovery

9. Troubleshooting

Berikut adalah masalah-masalah umum yang mungkin Anda temui saat memasang atau menjalankan kantorku, beserta solusinya. Sebagian besar masalah berkaitan dengan dependency yang kurang, konfigurasi API key, atau koneksi ke LLM provider.

9.1 ModuleNotFoundError

Jika Anda mendapatkan error ini saat import, kemungkinan besar provider-specific package belum diinstal. Kantorku menggunakan optional dependencies — setiap provider LLM dipasang terpisah. Pastikan Anda menginstal bracket notation yang benar saat menjalankan pip install.

```
# Error: ModuleNotFoundError: No module named 'anthropic'
# Solusi:
pip install kantorku[anthropic]

# Error: ModuleNotFoundError: No module named 'google.genai'
# Solusi:
pip install kantorku[google]
```

9.2 Provider Not Configured

Error ini muncul ketika worker mencoba menggunakan provider yang belum dikonfigurasi kredensialnya. Pastikan API key sudah diset di environment variable DAN section [providers.*] sudah ada di kantorku.toml dengan referensi \${ENV_VAR} yang benar.

```
# Error: Provider 'anthropic' not configured
# Solusi:
export ANTHROPIC_API_KEY="sk-ant-..."
# Verifikasi:
python3 -c "import os; print(os.environ.get('ANTHROPIC_API_KEY', 'NOT SET'))"
```

9.3 Ollama Connection Refused

Jika menggunakan Ollama dan mendapatkan connection error, pastikan Ollama server sedang berjalan dan model sudah di-pull. Ollama perlu dijalankan sebagai background service sebelum kantorku bisa menggunakannya.

```
# Cek apakah Ollama berjalan:
curl http://localhost:11434/api/tags

# Jalankan Ollama:
ollama serve

# Pull model yang dibutuhkan:
ollama pull llama3
```

9.4 DuckDB Ring1 Error

Jika DuckDB gagal membuat file database, pastikan directory tujuan ada dan memiliki permission yang benar. Kantorku secara default membuat file di `./data/`, yang mungkin belum ada di environment Anda.

```
# Error: Unable to open database file
# Solusi:
mkdir -p data
chmod 755 data
```

10. Arsitektur Overview

Kantorku memiliki arsitektur yang terdiri dari beberapa layer utama yang saling berinteraksi: Conductor sebagai CEO yang mengorkestrasi seluruh operasi, BriefingRoom sebagai ruang diskusi pra-eksekusi, WorkerHub sebagai lapisan komunikasi peer-to-peer antar worker, dan ContextPool sebagai sumber konteks proaktif yang siap sebelum worker mulai bekerja.

10.1 Flow Lengkap

Alur kerja kantorku dimulai dari pesan client, melalui negosiasi kontrak, briefing tim, prefetch konteks, eksekusi paralel, verifikasi, hingga selesai. Setiap langkah menghasilkan event yang dikirim ke Panel 2 untuk monitoring real-time. Berikut adalah urutan lengkapnya:

- Client kirim pesan ke Panel 1
- Conductor memahami pesan (bisa multi-turn clarification)
- Conductor menyiapkan kontrak (structured todo list)
- Client setuju / minta revisi
- BriefingRoom dibuka — workers speak_up paralel
- Context Pool prefetch konteks untuk setiap todo
- Conductor assign tasks ke workers (paralel jika bisa)
- Workers eksekusi dengan konteks yang sudah siap
- Workers bisa DM sesama atau request konteks tambahan
- Verifier memeriksa output (design + engineer paralel)
- Sentinel mencatat pelajaran
- Selesai — client menerima hasil

10.2 Perbandingan Framework

Kantorku memiliki perbedaan fundamental dibanding LangGraph dan CrewAI. Alih-alih menggunakan graph routing manual atau sequential process, kantorku menggunakan Conductor yang membuat keputusan cerdas berdasarkan konteks — mirip cara manager sungguhan mengorkestrasi timnya.

Fitur	LangGraph	CrewAI	kantorku
Orkestrasi	Manual routing	Process.sequential	Conductor (CEO)
Pre-execution	Tidak	Tidak	BriefingRoom
Worker comms	Tidak	Terbatas	WorkerHub DM

Proactive prefetch	Tidak	Tidak	ContextPool FIFO
Memory	External	Basic	3-Ring (hot/warm/cold)
Contract flow	Tidak	Tidak	Client <-> Manager
Real-time UI	Tidak	Tidak	Dual WebSocket

kantorku — karena kantor yang baik bukan yang paling canggih, tapi yang paling tahu siapa harus mengerjakan apa.